

Backend Node.js Internship Roadmap

Plan: Weeks 4-20

5-Month Backend Internship | 6 Working Hours per Day

Completed before this roadmap

Roadmap Overview

Monday to Thursday are used for learning and implementation. Friday is reserved for exercises covering the same week. The roadmap intentionally excludes RBAC, OAuth, API keys, modules, formal testing and separate documentation training.

Week	Main Focus	Friday Exercise Focus
4	Git, GitHub, HTTP, REST and Postman	Weekly Exercises
5	Node.js Fundamentals	Weekly Exercises
6	Express.js and REST API Development	Weekly Exercises
7	MongoDB and Mongoose	Weekly Exercises
8	Authentication with JWT and Refresh Tokens	Weekly Exercises
9	Pagination, Filtering, Search, Email and Cloudinary	Weekly Exercises
10	WebSockets with Socket.IO	Weekly Exercises
11	Cron Jobs and Scheduling	Weekly Exercises
12	React Basics for API Integration	Weekly Exercises
13	Mini Full-Stack Project - Express, MongoDB and React	Weekly Exercises
14	TypeScript Fundamentals	Weekly Exercises
15	NestJS Fundamentals	Weekly Exercises
16	NestJS, PostgreSQL and GraphQL	Weekly Exercises
17	Final Project - Planning and Foundation	Weekly Exercises
18	Final Project - Core Features	Weekly Exercises
19	Final Project - Frontend Integration and GraphQL	Weekly Exercises
20	Final Project - Stabilisation and Presentation	Final Exercises and Presentation

Week 4: Git, GitHub, HTTP, REST and Postman

Goal: Build the tooling and protocol foundation required for backend development.

Monday - Git Fundamentals	• Install and configure Git. • Understand repositories, working tree, staging area and commit history. • Practise init, clone, status, add, commit, log, diff and .gitignore, pull and push • Create a clean repository for the internship work. Create and switch branches. Practise merge and resolve a simple conflict.
Tuesday - Branching and GitHub Workflow	• Understand the request-response lifecycle. • Study methods: GET, POST, PUT, PATCH and DELETE. • Study status-code groups and commonly used codes. • Understand headers, request body, query parameters and path parameters. • Compare cookies and sessions at a basic level.
Wednesday - HTTP Fundamentals	• Understand resources, endpoints and statelessness. • Use correct resource naming and HTTP methods. • Create a REST design for a bookstore or task manager. • Create Postman requests, variables, environments and collections. • Test success and error responses from a public API.
Thursday - REST API Design and Postman	• Analyze real-world APIs. Read API Documentation. • Swagger Introduction. OpenAPI Basics. • How frontend consumes APIs. Common API Response Format. • Error Response Structure. Pagination Response Format (overview only). • Authentication Headers (overview only). •
Friday - Weekly Exercises	• Complete Git command exercises without copying steps. • Resolve a prepared merge conflict. • Identify correct HTTP methods and status codes for 15 scenarios. • Design endpoints for a small bookstore system. • Build and export a Postman collection containing CRUD requests.

Week 5: Node.js Fundamentals

Goal: Understand the Node.js runtime and build programs without a framework.

Monday - Runtime, npm and Modules	• Understand Node.js, V8 and server-side JavaScript. • Create and run scripts with Node. • Use npm, package.json, scripts and semantic versions. • Use CommonJS and ES Modules. • Create and import custom modules. • Manage File structure.
Tuesday - Core Modules	• Use fs for reading, writing, updating and deleting files. • Use path for safe file paths. • Use os and process for runtime information. • Work with environment variables using dotenv. • Create a file-based notes utility.
Wednesday - Event Loop and Asynchronous Node	• Understand call stack, callback queue and event loop. • Compare synchronous and asynchronous file operations. • Use callbacks, promises and async/await in Node. • Handle rejected promises and uncaught errors correctly. • Observe execution order through small experiments.
Thursday - Native HTTP Server	• Create a server using the built-in http module. • Read method, URL, headers and request body. • Return JSON responses and status codes. • Create basic routes without Express. • Organise server logic into small modules.
Friday - Weekly Exercises	• Build a CLI utility that reads and writes JSON data. • Convert callback-based file operations to promises. • Predict event-loop output for several code samples. • Build a native Node server with at least five routes. • Fix prepared bugs involving modules, paths and asynchronous code.

Week 6: Express.js and REST API Development

Goal: Build structured REST APIs with Express.

Monday - Express Setup and Routing	• Create an Express project and use nodemon. • Create routers and route handlers. • Use route parameters, query parameters and request bodies. • Return consistent JSON responses. • Create basic CRUD routes for one resource.
Tuesday - Middleware	• Understand middleware execution order. • Use built-in and third-party middleware. • Create request logging and request-timing middleware. • Create middleware for validation checks. • Understand application-level and router-level middleware.
Wednesday - Controllers, Services and MVC	• Separate routes, controllers and services. • Understand responsibilities of each layer. • Create reusable service functions. • Apply a clean folder structure. • Refactor a single-file API into modules.
Thursday - Validation and Error Handling	• Validate required fields and formats. • Create custom error objects. • Use centralized error-handling middleware. • Handle 404 and async errors. • Return appropriate status codes and messages.
Friday - Weekly Exercises	• Build a complete in-memory CRUD API. • Add custom middleware and request validation. • Refactor the API into routes, controllers and services. • Add centralized error handling. • Test all success and failure cases in Postman.

Week 7: MongoDB and Mongoose

Goal: Persist Express APIs with MongoDB and model data using Mongoose.

Monday - MongoDB Fundamentals	• Understand databases, collections, documents and BSON. • Use MongoDB Compass or Atlas. • Perform insert, find, update and delete operations. • Use filters, projections and sorting. • Understand embedded documents and references.
Tuesday - Mongoose Schemas and Models	• Connect Express to MongoDB. • Create schemas and models. • Use data types, defaults, required fields and enums. • Use timestamps and schema methods. • Understand model validation.
Wednesday - CRUD with Mongoose	• Create repository/service functions with Mongoose. • Use find, findById, update and delete operations. • Handle invalid IDs and missing documents. • Use select and lean where appropriate. • Connect CRUD endpoints to MongoDB.
Thursday - Relationships and Aggregation Basics	• Use references and populate. • Compare embedding with referencing. • Create simple aggregation pipelines. • Use match, project, group, sort and lookup at an introductory level. • Discuss basic indexing and uniqueness.
Friday - Weekly Exercises	• Design schemas for a small course or task application. • Build MongoDB-backed CRUD endpoints. • Create one referenced relationship and populate it. • Write five aggregation exercises. • Fix validation, duplicate-data and invalid-ID issues.

Week 8: Authentication with JWT and Refresh Tokens

Goal: Implement secure authentication using passwords, access tokens and refresh tokens.

Monday - User Registration and Password Hashing	• Create a user schema. • Validate registration input. • Hash passwords with bcrypt. • Prevent duplicate email registration. • Never return password hashes in API responses.
Tuesday - Login and Access Tokens	• Verify credentials. • Create JWT access tokens. • Understand token payload, secret and expiry. • Create login responses. • Use environment variables for secrets.
Wednesday - Protected Routes	• Read Bearer tokens from headers. • Verify JWTs in middleware. • Attach the authenticated user to the request. • Protect profile and private routes. • Handle missing, invalid and expired tokens.
Thursday - Refresh Token Flow	• Understand why refresh tokens are used. • Issue and verify refresh tokens. • Create a refresh endpoint. • Store or rotate refresh tokens using a simple safe approach. • Implement logout by invalidating the stored refresh token.
Friday - Weekly Exercises	• Build registration and login endpoints. • Create access-token and refresh-token flows. • Protect at least two routes. • Handle expired and invalid tokens. • Test the complete flow in Postman, including logout.

Week 9: Pagination, Filtering, Search, Email and Cloudinary

Goal: Add practical features commonly required by real applications.

Monday - Pagination	• Implement page and limit query parameters. • Calculate skip and total pages. • Return pagination metadata. • Apply maximum page-size limits. • Handle empty and out-of-range pages.
Tuesday - Filtering, Sorting and Search	• Build filters from approved query parameters. • Implement text search using regular expressions. • Implement sorting and field selection. • Combine pagination, filtering and search safely. • Avoid passing arbitrary query objects directly to MongoDB.
Wednesday - Email Service	• Configure Nodemailer. • Create a reusable email service. • Send welcome and password-reset emails. • Use HTML templates with dynamic values. • Handle email failures without crashing the API.
Thursday - Cloudinary	• Upload images using multipart form data. • Configure Multer and Cloudinary. • Store Cloudinary URL and public ID. • Replace and delete uploaded images. • Validate file type and size.
Friday - Weekly Exercises	• Add pagination metadata to a list endpoint. • Create combined filter, sort and search queries. • Send a test welcome email. • Upload, replace and delete an image through Cloudinary. • Integrate all features into an existing Express API.

Week 10: WebSockets with Socket.IO

Goal: Build real-time communication features with Socket.IO.

Monday - WebSocket Fundamentals	• Compare HTTP polling with persistent WebSocket connections. • Understand client-server connection lifecycle. • Set up Socket.IO on the backend and a simple client. • Handle connection and disconnection events. • Emit and listen to custom events.
Tuesday - Broadcasting and Rooms	• Emit to one socket, all sockets and all except sender. • Create and join rooms. • Send room-specific messages. • Use acknowledgements for event responses. • Handle invalid event payloads.
Wednesday - Real-Time Chat	• Create chat message events. • Track connected users in memory. • Add typing indicators. • Add online/offline status. • Store chat messages in MongoDB at a basic level.
Thursday - Notifications and Authentication	• Create real-time notification events. • Authenticate socket connections using JWT. • Map users to sockets. • Handle reconnection and duplicate connections. • Separate socket handlers into modules.
Friday - Weekly Exercises	• Build a room-based chat server. • Implement typing and presence events. • Create a private notification event. • Protect the socket connection with JWT. • Debug prepared connection, event-name and room issues.

Week 11: Cron Jobs and Scheduling

Goal: Automate recurring and scheduled backend tasks.

Monday - Scheduling Fundamentals	• Understand recurring jobs and delayed tasks. • Learn cron-expression structure. • Use node-cron or Nest schedule concepts at an introductory level. • Create jobs that run at fixed intervals. • Prevent jobs from running during local development when not required.
Tuesday - Automated Email Jobs	• Send daily or weekly summary emails. • Query records due for notification. • Avoid sending duplicate emails. • Log job start, completion and failures. • Use configurable schedules from environment variables.
Wednesday - Database Maintenance Jobs	• Create jobs for expired-token cleanup. • Archive or update expired records. • Process pending records in safe batches. • Understand idempotency for scheduled jobs. • Handle partial failures.
Thursday - Scheduling Design	• Choose between request-time logic and scheduled jobs. • Create job service modules. • Add basic locking/status fields to avoid duplicate processing. • Test cron logic by using short development schedules. • Discuss timezone considerations.
Friday - Weekly Exercises	• Write cron expressions for 10 scheduling requirements. • Build an automated reminder-email job. • Build an expired-record cleanup job. • Make a job idempotent. • Fix prepared duplicate-run and timezone issues.

Week 12: React Basics for API Integration

Goal: Learn only the frontend knowledge needed to consume backend APIs.

Monday - React Setup and Components	• Create a React app with Vite. • Understand JSX, components and props. • Create a simple layout and reusable input/button components. • Render lists with keys. • Keep the UI simple and functional.
Tuesday - State, Events and Forms	• Use useState. • Handle input and button events. • Create controlled forms. • Perform basic client-side validation. • Display validation messages.
Wednesday - Routing and API Calls	• Set up React Router. • Create list, create, edit and login pages. • Use fetch or Axios. • Use useEffect for initial data loading. • Handle loading, empty and error states.
Thursday - Authentication Integration	• Store and send access tokens. • Create an Axios instance or shared API helper. • Protect simple frontend routes. • Handle token expiry and refresh requests. • Connect forms to the existing backend.
Friday - Weekly Exercises	• Create a CRUD interface for an existing API. • Build login and protected profile pages. • Add loading and error states. • Integrate pagination or search from the backend. • Fix prepared form and API-integration bugs.

Week 13: Mini Full-Stack Project - Express, MongoDB and React

Goal: Combine the learned Express stack into a small complete application.

Monday - Planning and Setup	• Select a small project such as task manager, appointment system or expense tracker. • Define entities and MongoDB schemas. • List REST endpoints and frontend screens. • Create backend and frontend projects. • Divide work into small tasks.
Tuesday - Backend Implementation	• Build authentication. • Build core CRUD endpoints. • Add validation and centralized errors. • Add pagination, filtering or search. • Add one email or Cloudinary feature.
Wednesday - Frontend Implementation	• Build pages and forms. • Connect list and CRUD screens. • Add login and protected screens. • Handle loading, errors and empty states. • Keep styling simple.
Thursday - Integration and Fixes	• Test complete user flows. • Fix frontend-backend contract mismatches. • Handle token refresh. • Review folder structure and repeated code. • Prepare a working demo.
Friday - Weekly Exercises	• Demonstrate the full application. • Complete a mentor-provided change request. • Fix at least three bugs found during review. • Explain the request flow from React to MongoDB. • Refactor one weak part of the project after feedback.

Week 14: TypeScript Fundamentals

Goal: Prepare interns for NestJS by introducing practical TypeScript.

Monday - Types and Configuration	• Set up TypeScript and tsconfig. • Use primitive types, arrays, tuples and enums. • Understand inference and annotations. • Use union and literal types. • Compile and run TypeScript.
Tuesday - Interfaces, Types and Functions	• Type function parameters and returns. • Use interfaces and type aliases. • Use optional and readonly properties. • Create reusable request and response types. • Use narrowing.
Wednesday - Classes and Object-Oriented TypeScript	• Create classes and constructors. • Use public, private and protected members. • Use inheritance and abstract classes. • Implement interfaces. • Understand dependency-oriented class design.
Thursday - Generics and Express Migration	• Create simple generic functions and response types. • Use Partial, Pick, Omit and Record. • Type Express requests, responses and middleware. • Convert a small Express module from JavaScript to TypeScript. • Fix common compiler errors.
Friday - Weekly Exercises	• Complete type-conversion exercises. • Create interfaces for an API domain. • Build generic pagination response types. • Migrate one Express CRUD module to TypeScript. • Resolve prepared TypeScript errors without using any.

Week 15: NestJS Fundamentals

Goal: Learn NestJS structure, dependency injection and API development patterns.

Monday - Project Structure and Modules	• Create a project using Nest CLI. • Understand modules, controllers and providers. • Generate modules and resources. • Understand decorators at a practical level. • Configure environment variables.
Tuesday - Controllers and Services	• Create routes with Nest decorators. • Use params, queries and request bodies. • Create injectable services. • Understand constructor dependency injection. • Keep controllers thin.
Wednesday - DTOs and Validation	• Create DTO classes. • Use class-validator and class-transformer. • Use ValidationPipe globally. • Create create and update DTOs. • Return consistent validation errors.
Thursday - CRUD Module and Error Handling	• Build a complete in-memory CRUD module. • Use built-in HTTP exceptions. • Create reusable response and exception patterns. • Organise features by module. • Compare Express structure with NestJS structure.
Friday - Weekly Exercises	• Create a new NestJS resource from scratch. • Build CRUD routes with DTO validation. • Use dependency injection correctly. • Add error handling for missing records. • Refactor a poorly structured Nest module.

Week 16: NestJS, PostgreSQL and GraphQL

Goal: Persist NestJS data in PostgreSQL and introduce GraphQL after REST fundamentals.

Monday - PostgreSQL and TypeORM Setup	• Understand tables, rows, primary keys and foreign keys. • Configure PostgreSQL with NestJS and TypeORM. • Create entities and repositories. • Run basic CRUD operations. • Use environment-based database configuration.
Tuesday - Relations and Nest Authentication	• Create one-to-many and many-to-one relations. • Use joins and relation loading through TypeORM. • Implement JWT authentication in NestJS. • Create an auth guard and protected route. • Implement refresh-token logic in the NestJS project.
Wednesday - GraphQL Fundamentals	• Compare REST and GraphQL. • Understand schema, types, queries and mutations. • Configure GraphQL in NestJS. • Create object types and input types. • Build simple queries and mutations.
Thursday - GraphQL Resolvers with PostgreSQL	• Create resolvers and connect them to services. • Use arguments and nested fields. • Reuse the same service layer for REST and GraphQL where practical. • Handle GraphQL errors and validation. • Create a small GraphQL CRUD flow backed by PostgreSQL.
Friday - Weekly Exercises	• Create related PostgreSQL entities. • Build REST CRUD using TypeORM. • Protect one NestJS route with JWT. • Build GraphQL queries and mutations for one resource. • Explain when REST or GraphQL is more suitable for a given case.

Week 17: Final Project - Planning and Foundation

Goal: Plan the final project and establish the backend foundation.

Monday - Scope and User Flows	• Confirm the project idea and keep the scope achievable. • Define users and main flows. • List required modules and features. • Identify where JWT, email, uploads, WebSockets, cron or GraphQL will be used. • Divide work among the five interns.
Tuesday - Database and API Design	• Design PostgreSQL entities and relationships. • Define REST and optional GraphQL operations. • Agree request and response formats. • Create initial project structure. • Prepare environment configuration.
Wednesday - Authentication Foundation	• Implement user entity and registration. • Implement login, access tokens and refresh tokens. • Add protected routes. • Create base validation and error handling. • Review authentication flow as a team.
Thursday - Core Modules	• Create initial modules, controllers, services and entities. • Implement first core CRUD resource. • Create shared utilities only where necessary. • Resolve merge conflicts through pull requests. • Keep code consistent across team members.
Friday - Weekly Exercises	• Each intern explains assigned modules and entity relations. • Complete a live coding task from the project domain. • Review and correct authentication edge cases. • Resolve one mentor-created Git conflict. • Apply review feedback before the next week.

Week 18: Final Project - Core Features

Goal: Build the main application features and integrations.

Monday - Remaining Core Modules	• Implement remaining entities and relations. • Complete CRUD services and controllers. • Add filtering, search and pagination. • Review database queries for unnecessary repetition. • Keep all changes in focused pull requests.
Tuesday - Email and Cloudinary	• Implement the required email flow. • Implement image or document upload through Cloudinary. • Validate files and handle replacement/deletion. • Persist upload metadata. • Handle integration errors cleanly.
Wednesday - WebSockets	• Implement the selected real-time feature. • Authenticate socket connections. • Use rooms or user-specific channels. • Store required real-time data. • Handle disconnect and reconnection cases.
Thursday - Cron and Scheduling	• Implement at least one meaningful scheduled job. • Make the job safe against duplicate processing. • Add logs for successful and failed runs. • Use configurable schedules. • Review timezone and retry behaviour.
Friday - Weekly Exercises	• Demo each completed integration. • Complete a mentor-provided feature change. • Fix integration issues discovered in review. • Explain how WebSocket and cron flows work. • Refactor duplicated service logic.

Week 19: Final Project - Frontend Integration and GraphQL

Goal: Connect the simple React frontend and complete all major user flows.

Monday - Frontend Structure and Authentication	• Create routing and basic page structure. • Build login and protected navigation. • Configure shared API calls. • Handle access-token refresh. • Keep design simple and usable.
Tuesday - Core Screens	• Build list, details and form screens. • Connect CRUD operations. • Add validation feedback. • Add pagination, filtering and search controls. • Handle loading, errors and empty results.
Wednesday - Uploads and Real-Time UI	• Connect Cloudinary upload flows. • Display uploaded media. • Connect Socket.IO notifications or chat. • Handle connection status. • Prevent duplicate event rendering.
Thursday - GraphQL Integration and Full Flow Review	• Consume selected GraphQL queries or mutations where included. • Verify REST and GraphQL behaviour. • Walk through all key user flows. • Find contract mismatches and edge cases. • Create a final bug list.
Friday - Weekly Exercises	• Each intern completes one frontend-backend integration task. • Fix assigned bugs from the full-flow review. • Explain token refresh and API-call flow. • Debug a prepared real-time UI issue. • Demonstrate all assigned features.

Week 20: Final Project - Stabilisation and Presentation

Goal: Finish, review and present the complete project.

Monday - Bug Fixing	• Prioritise functional bugs. • Fix authentication and validation edge cases. • Fix incorrect database relations or queries. • Fix WebSocket and scheduled-job issues. • Retest complete user flows.
Tuesday - Code Quality Review	• Remove dead and duplicated code. • Improve naming and folder consistency. • Check environment configuration. • Review error messages and response formats. • Ensure each intern can explain their contributed code.
Wednesday - Final Integration	• Merge remaining pull requests carefully. • Run the full application from a clean setup. • Verify email, Cloudinary, WebSockets, cron and GraphQL features. • Fix final integration problems. • Prepare sample users and data for demonstration.
Thursday - Demo Preparation	• Prepare the demonstration flow. • Assign presentation sections. • Practise explaining architecture and technical decisions. • Prepare backup screenshots or data. • Run a complete rehearsal.
Friday - Final Exercises and Presentation	• Complete a short individual live-coding exercise. • Answer technical questions about the project. • Present the application as a team. • Demonstrate each major backend feature. • Receive final mentor feedback.